

What is Regression?

Predicting a continuous number from input features

Regression is a supervised learning technique where the goal is to predict a continuous numerical value (output) from one or more input features. Unlike classification (which predicts a category), regression predicts a quantity — a house price, tomorrow's temperature, or an employee's expected salary

Input Features (X)	Output to Predict (y)	Real Example
House area, location	Sale price (\$)	Zillow / Magicbricks
Years of experience	Annual salary	LinkedIn salary tool
Date, humidity, wind	Temperature (°C)	Weather forecast
Rainfall, soil type	Crop yield (tonnes/acre)	AgriTech models
Ad spend	Revenue generated	Marketing analytics

The main goal of regression is:

Understand the relationship between input and output Predict future values using that relationship

The fitted line tries to stay as close as possible to all the data points together, not just one point. Once the model learns the pattern, we can give it a new input value and it will estimate the corresponding output

Regression is a machine learning technique used to predict continuous numerical values based on patterns found in existing data.

Linear regression is a type of supervised machine-learning algorithm that learns from the labelled datasets and maps the data points with most optimized linear functions which can be used for prediction on new datasets. It assumes that there is a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line.

Introduction to Linear Regression



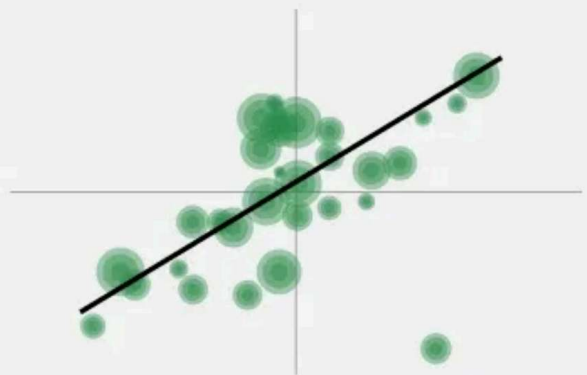
Simple model for predicting continuous values.



It finds the relationship between input features and output.



It is used in forecasting and predicting salaries based on experience.



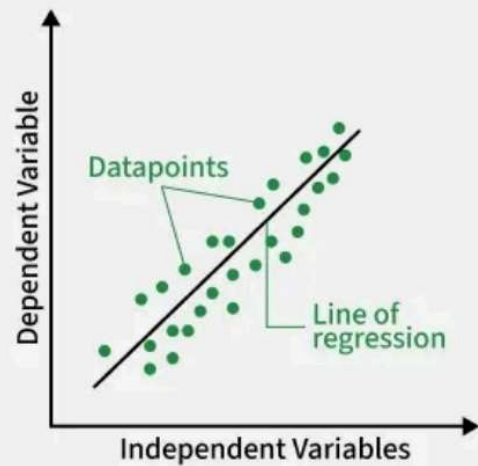
How Does Linear Regression Work?



Finds the best-fitting line by minimizing prediction errors (least squares method)

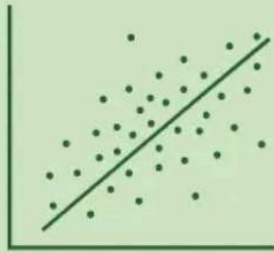


It calculates coefficients for variables that minimize the error in predictions.



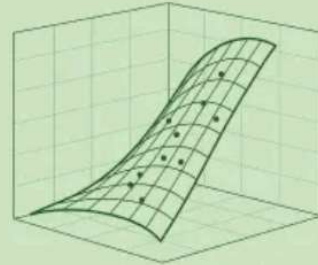
Types of Linear Regression

Simple Linear Regression



Predicts the dependent variable using a single independent variable.

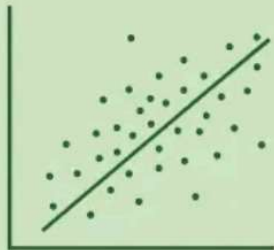
Multiple Linear Regression



Uses two or more independent variables to predict the dependent variable.

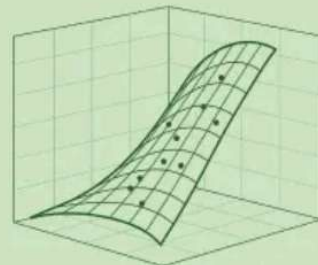
Types of Linear Regression

Simple Linear Regression



Predicts the dependent variable using a single independent variable.

Multiple Linear Regression



Uses two or more independent variables to predict the dependent variable.

For example we want to predict a student's exam score based on how many hours they studied. We observe that as students study more hours, their scores go up.

In the example of predicting exam scores based on hours studied. Here

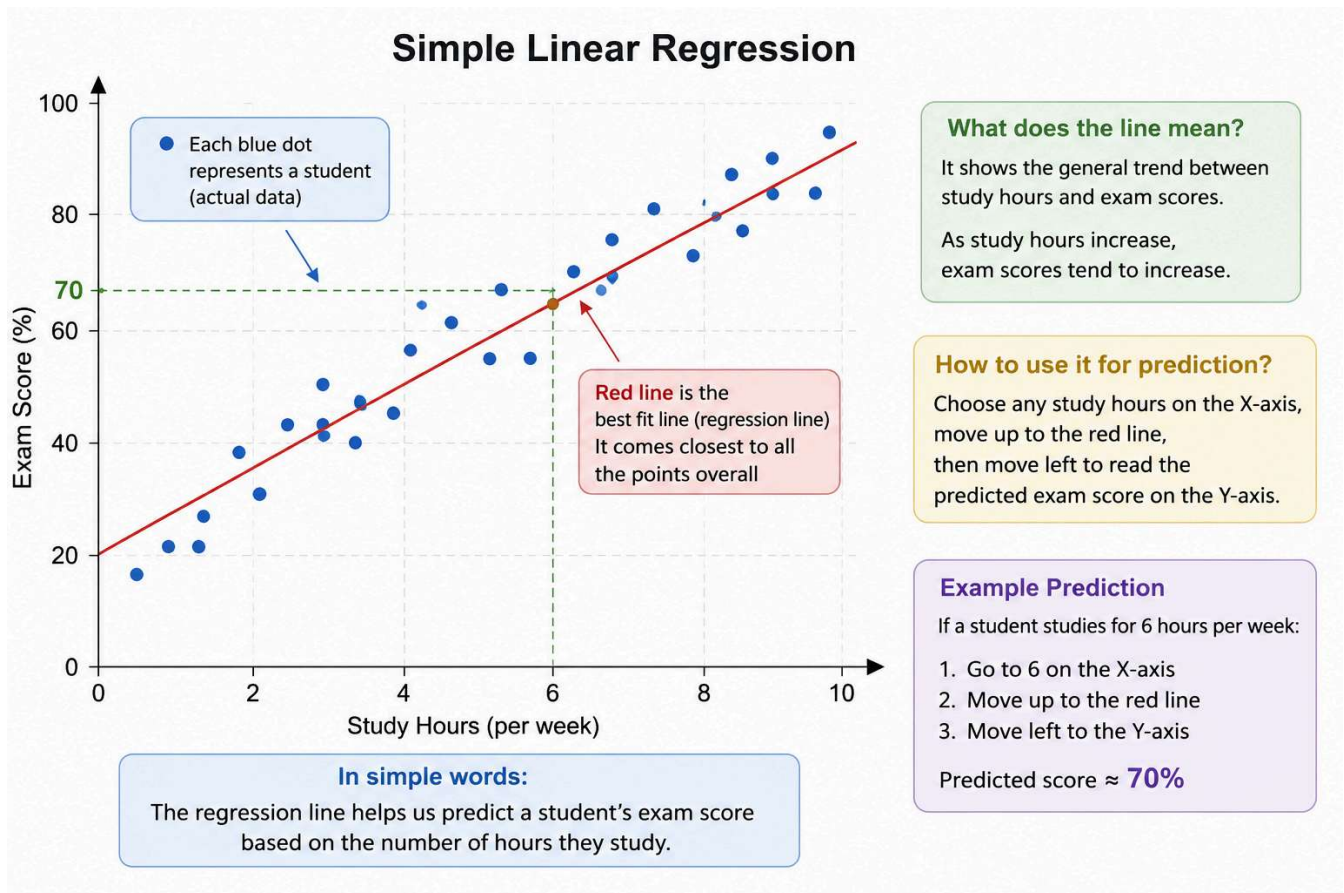
Independent variable (input): Hours studied because it's the factor we control or observe.

Dependent variable (output): Exam score because it depends on how many hours were studied.

We use the independent variable to predict the dependent variable.

THE BEST FIT LINE IN LINEAR REGRESSION

In linear regression, the best-fit line is the straight line that most accurately represents the relationship between the independent variable (input) and the dependent variable (output). It is the line that minimizes the difference between the actual data points and the predicted values from the model.



Imagine plotting study hours on the X-axis and exam scores on the Y-axis. Each student is a dot. A regression model draws the single straight line that comes closest to all the dots simultaneously. That line lets us predict any student's score just by knowing their study hours.

$$y = m \cdot x + c$$

y = output | x = input | m = slope | c = intercept

```
import numpy as np

# Input data
hours = np.array([1,2,3,4,5,6,7,8,9,10])
scores = np.array([60,70,80,90,100,110,120,130,140,150])

# Linear Regression tries to find the BEST FIT LINE
# Equation of line:
# score = m * hours + c
#
# m = slope of the line
# c = intercept of the line

# Manual understanding of slope:
# slope = change in y / change in x
# slope  $\approx$  (150 - 60) / (10 - 1)
# slope  $\approx$  90 / 9
# slope  $\approx$  10

# Intercept:
# c = y - mx
# c = 60 - (10 * 1)
# c = 50

# Final best fit equation:
# predicted_score = 10 * hours + 50

# Predict score for 5 study hours
predicted_score = 10 * 5 + 50
```

```
print("Best Fit Line Equation:")
print("score = 10 * hours + 50")

print("\nFor 5 hours → predicted score:", predicted_score)
```

For 5 hours → predicted score: 68.4

Each value in hours represents study hours.

Each value in scores represents exam marks.

Linear Regression finds the best fit line that represents the relationship between these two variables.

The slope (m) tells how much the score increases when study hours increase by 1.

The intercept (c) is the starting point of the line.

```
import numpy as np
import matplotlib.pyplot as plt

# Original data
hours = np.array([1,2,3,4,5,6,7,8,9,10])
scores = np.array([60,70,80,90,100,110,120,130,140,150])

# Best Fit Line Equation
# y = 10x + 50

predicted_scores = 10.41 * hours + 48.52

# Create graph
plt.figure(figsize=(8,6))

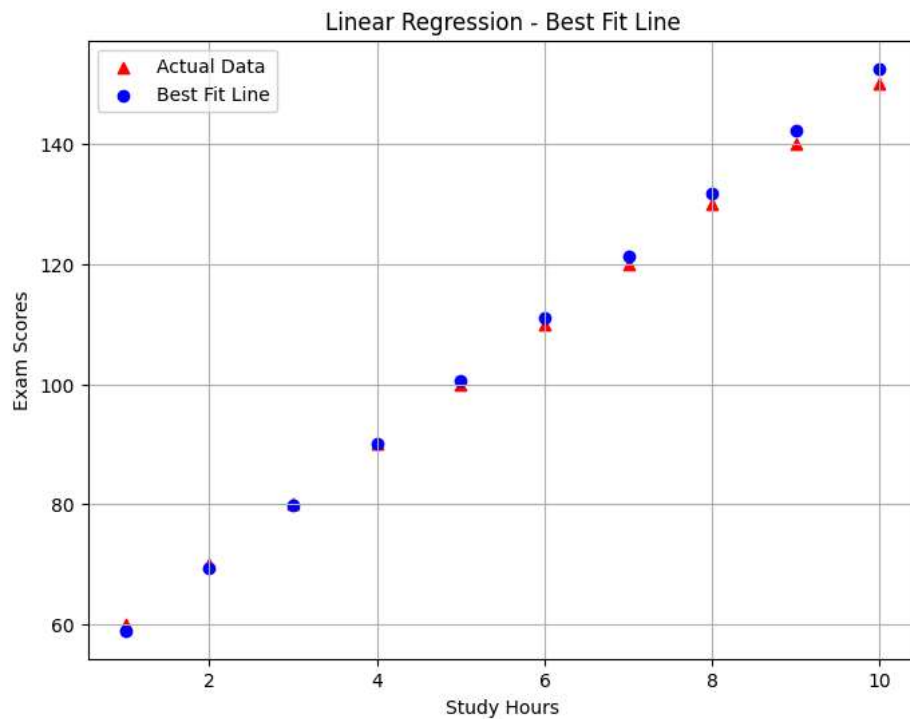
# Actual data points
plt.scatter(hours, scores, label="Actual Data", marker="^", color="red")

# Best fit regression line
plt.scatter(hours, predicted_scores, label="Best Fit Line", color="blue")

# Labels and title
plt.xlabel("Study Hours")
plt.ylabel("Exam Scores")
plt.title("Linear Regression - Best Fit Line")

# Show legend
plt.legend()

# Show graph
plt.grid(True)
plt.show()
```



HOW THE MODEL LEARNS

A regression model learns by adjusting its slope (m) and intercept (c) so that predictions become as accurate as possible. This happens by minimising a loss function — a number that measures how wrong the model currently is.

ERROR/PREDICTION

$$\text{Error} = \text{Predicted} - \text{Actual}$$

Positive → over-predicted | Negative → under-predicted

By above calculations we will get equation as

$$y = mx + c$$

m = slope

x = hours

c = interception

Suppose:

Study Hours = 5 Actual Score = 100

If the model currently uses:

$$m = 8$$

$$c = 40$$

Then prediction becomes:

$$y = 8(5) + 40 = 80$$

So:

Actual Score = 100

Predicted Score = 80

The model is wrong by:

$$\text{Error} = \text{Actual} - \text{Predicted} = 100 - 80 = 20$$

This difference is called Error.

Mean Squared Error (MSE)

$$\text{MSE} = (1/n) \times \sum (y_{\text{pred}} - y_{\text{actual}})^2$$

n = number of samples | Σ = sum over all samples

The model checks errors for all data points.

Instead of simply adding errors, we square them:

Negative values become positive Large mistakes get punished more

$y_{\text{actual}} \rightarrow$ real value

$y_{\text{predicted}} \rightarrow$ model prediction

$n \rightarrow$ number of data points

for checking mse values we need to run above example for multiple times to check whether it is working properly or not

if we take new data as

Step-1: find predicted values

hours[5,8,7]

Score[100, 120, 90]

Student 1

Study Hours = 5

Prediction:

$y = 8(5) + 40 = 80$

So:

Actual = 100

Predicted = 80

calculate for remaining students

Step 2 — Calculate Error

Error=Actual–Predicted

Student 1

$100 - 80 = 20$

Calculate for remaining students

Step 3 — Square the Error

Formula:

Error 2 =(Actual–Predicted) 2

Student 1

$20^2 = 400$

calculate for remaining students

Actual	Predicted	Error	Error ²
100	80	20	400
120	104	16	256

Actual	Predicted	Error	Error ²
90	96	-6	36

GRADIENT DESCENT

Gradient Descent is the algorithm that actually updates m and c to reduce MSE. Imagine you're blindfolded on a hilly landscape and want to reach the lowest point (minimum MSE). You feel the slope under your feet and take a small step downhill. Repeat until you can't go lower.

Gradient Descent works like this:

1. Start with random values of m and c
2. Calculate predictions
3. Compute error and MSE
4. Adjust m and c
5. Repeat again and again until error
6. becomes very small

Imagine you are trying to find the coldest spot in a dark room.

You cannot see anything, but you can feel the temperature.

If you move in one direction and it becomes colder, you continue moving that way. If it becomes warmer, you change direction.

You keep taking small steps again and again until you reach the coldest place in the room.

Real Life	Gradient Descent
Dark room	Machine learning problem
Coldest spot	Minimum error
Feeling temperature	Calculating loss/error
Small movements	Updating parameters
Reaching coldest area	Finding best fit model

Formula

$$\frac{\partial L}{\partial m} = \frac{-2}{n} \sum x(y - \hat{y})$$

Where:

- **L** = loss function (MSE)
- **x** = input values
- **y** = actual values
- **$\hat{y}$** = predicted values
- **error = y - $\hat{y}$**

LETS TRY WITH EXAMPLE CONSIDER ALL VALUES THAT ARE CALCULATED AS ZERO .

```
import numpy as np

# -----
# Dataset
# -----
# x = study hours
# y = actual scores

x = np.array([1,2,3,4,5], dtype=float)
y = np.array([60,70,80,90,100], dtype=float)

# -----
# Initial values of m and c
# The model initially knows nothing.
# -----
m = 0
c = 0

# Learning rate
learning_rate = 0.01

# Number of iterations
epochs = 1000

# Total number of data points
n = len(x)

# -----
# Gradient Descent
# -----

for i in range(epochs):
```

```

# Predicted values
y_pred = m * x + c

# Error
error = y - y_pred

# Mean Squared Error (MSE)
mse = np.mean(error ** 2)

# Gradients
dm = (-2/n) * np.sum(x * error)#How much the slope (m) should change
dc = (-2/n) * np.sum(error)#How much the intercept (c) should change

# Update m and c
m = m - learning_rate * dm
c = c - learning_rate * dc

# Print progress every 100 iterations
if i % 100 == 0:
    print(f"Epoch {i}")
    print(f"MSE = {mse:.2f}")
    print(f"m = {m:.2f}, c = {c:.2f}")
    print("-----")

# Final equation
print("\nFinal Best Fit Line:")
print(f"y = {m:.2f}x + {c:.2f}")

```

```

Epoch 0
MSE = 6600.00
m = 5.20, c = 1.60
-----
Epoch 100
MSE = 177.90
m = 18.63, c = 18.84
-----
Epoch 200
MSE = 90.37
m = 16.15, c = 27.79
-----
Epoch 300
MSE = 45.90
m = 14.38, c = 34.17
-----
Epoch 400
MSE = 23.32
m = 13.12, c = 38.72
-----
Epoch 500
MSE = 11.84
m = 12.23, c = 41.96
-----
Epoch 600
MSE = 6.02
m = 11.59, c = 44.27
-----
Epoch 700
MSE = 3.06
m = 11.13, c = 45.92
-----
Epoch 800
MSE = 1.55
m = 10.81, c = 47.09
-----
Epoch 900
MSE = 0.79
m = 10.57, c = 47.93
-----

Final Best Fit Line:
y = 10.41x + 48.52

```

Building Models with sklearn

Train / Test Split — Why it matters If you train and test on the same data, the model's score is dishonest — it has memorised the answers.

We always keep a portion of data unseen during training so the final score reflects real-world performance

■ Golden Rule

80% of data → Training set (model learns from this). 20% of data → Test set (we evaluate on this).

Never let test data touch the training phase!

```
from sklearn.model_selection import train_test_split
import numpy as np
# Example: salary prediction from experience
experience = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15])
salary = np.array([35,38,42,48,55,60,65,70,76,82,87,92,96,100,105])
X = experience.reshape(-1, 1) # sklearn expects 2D array for features
y = salary
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2, # 20% for testing
    random_state=42 # ensures same split every run
)
print(f"Training samples : {len(X_train)}") # 12
print(f"Testing samples : {len(X_test)}") # 3
```

```
Training samples : 12
Testing samples : 3
```

Fitting and Predicting

```
from sklearn.linear_model import LinearRegression
# 1. Create the model object
model = LinearRegression()

# 2. Train (fit) on training data
model.fit(X_train, y_train)

# 3. Inspect what the model learned
print(f"Slope (coefficient) : {model.coef_[0]:.2f}") # slope=coef
print(f"Intercept : {model.intercept_: .2f}")

# 4. Predict on TEST data (unseen!)
y_pred = model.predict(X_test)

# 5. See predictions vs actual
for actual, predicted in zip(y_test, y_pred):
    print(f"Actual: {actual:>5} | Predicted: {predicted:>7.1f} | "
          f"Error: {predicted - actual:+.1f}")
```

```
Slope (coefficient) : 5.22
Intercept : 27.94
Actual:    82 | Predicted:    80.2 | Error: -1.8
Actual:    92 | Predicted:    90.6 | Error: -1.4
Actual:    35 | Predicted:    33.2 | Error: -1.8
```

Evaluation Metrics

After predicting, we need to **measure quality** using three main metrics:

Metric	Formula	Meaning	Ideal
R ² Score	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	% of variance explained (0–1)	Close to 1.0
MAE	$\text{mean}(y_{\text{pred}} - y_{\text{actual}})$	Average absolute error	Close to 0
RMSE	$\sqrt{\text{mean}((y_{\text{pred}} - y_{\text{actual}})^2)}$	Like MAE but penalises big errors	Close to 0

Metric	What it tells	Best for
R ² Score	How well model explains data	Model performance
MAE	Average error	Simple interpretation
RMSE	Penalizes large errors	Detecting big mistakes

calculating all three metrics

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

y_actual = np.array([500, 300, 700, 450, 600])
y_pred = np.array([480, 320, 690, 460, 590])
#How much of the variation in data is explained by the model
r2 = r2_score(y_actual, y_pred)
#Average of absolute errors (ignores + / - signs)
```